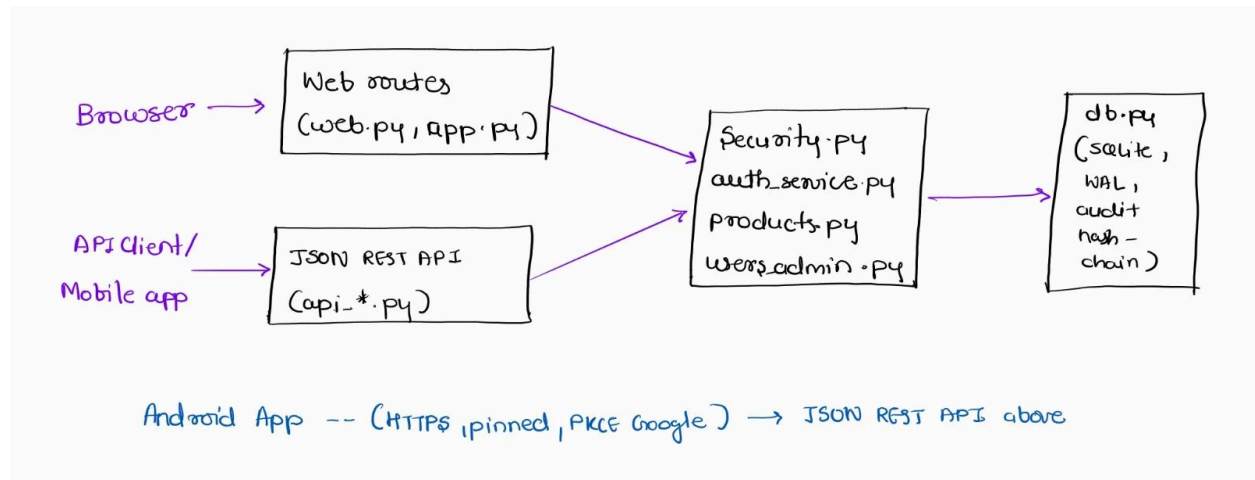


Threat Model - Extended Product & Product Search

1. System overview

Three client interfaces (server-rendered Web, JSON REST API, native Android Mobile) share one Flask backend and one SQLite database. All three enforce identical policy because they call the same centralized functions: security.py (RBAC/JWT/CSRF/crypto), auth_service.py (login/MFA/password reset/Google), products.py (role-scoped search, price protection), users_admin.py (user administration).



2. Assets

Asset	Why it matters
User credentials (password hashes, MFA secrets)	Compromise = account takeover across all 3 interfaces
Product price/cost/stock data	Direct financial impact; race conditions = lost revenue or inventory corruption
JWT access/refresh tokens (API/mobile) & session cookies (web)	Bearer credentials; theft = impersonation
Audit log (hash-chained)	Only source of truth for "who did what" during an incident
Google OAuth tokens/id_tokens	Federated identity; a forged/unverified one = account takeover without ever knowing a password

3. Actors

- **Customer** - browses/searches active listings only.
- **Vendor** - manages their own product listings (price/stock/cost visible).
- **Admin** - full product + user administration, audit log access.
- **External attacker** (unauthenticated) - network-level, credential stuffing, forged tokens, MITM on mobile.
- **Malicious/compromised insider** (any authenticated role) - tries to exceed their role's permissions (IDOR, privilege escalation).

4. Trust boundaries

1. Browser / mobile device <-> backend (network boundary - TLS required in any real deployment; enforced for mobile via `network_security_config.xml` pinning, and via `LAB_HTTPS/SESSION_COOKIE_SECURE` for the web session cookie).
2. Backend <-> Google (OAuth/OIDC) - the backend must independently verify every claim Google-issued tokens carry rather than trusting whatever the client forwards.
3. Backend <-> SQLite file - single-writer-at-a-time trust boundary; this is exactly where the price-protection concurrency controls live.
4. Role boundary inside the backend itself - customer/vendor/admin code paths share the same database and the same session/token format, so the RBAC layer (not network topology) is what keeps them apart.

5. STRIDE per component

Web routes (`web.py`, `app.py`)

- **Spoofing**: session fixation/CSRF → mitigated by `SESSION_COOKIE_HTTPONLY` + `SameSite=Lax` + synchronizer-token CSRF (`security.csrf_protect`) on every state-changing POST.
- **Tampering**: form fields (role, vendor_id, price) → server never trusts client-supplied ownership/role fields; `products.py/users_admin.py` re-derive them from the session/DB row.
- **Repudiation**: every login, price change, and admin action is written to the hash-chained audit log (`db.log_event`).
- **Information disclosure**: role-scoped field visibility (`products.visible_fields`) stops a customer from ever receiving cost/ stock/audit fields in an HTTP response, not just hiding them in the UI.
- **DoS**: Flask-Limiter caps login/MFA/price endpoints.
- **Elevation of privilege**: `security.role_required` / IDOR ownership checks in `products._owns_or_admin`.

JSON REST API / Mobile (`api_*.py`)

Same threats as Web, plus:

- **Token theft** (mobile): mitigated by `EncryptedSharedPreferences` (`TokenStore`), `allowBackup="false"`, short-lived access tokens (15 min), refresh-token rotation with revocation on reuse detection.
- **Forged Google identity**: mitigated by independent server-side verification (`security.verify_google_id_token_mobile`, `validate_google_userinfo`) - audience pinned to a mobile-only OAuth client id, signature/issuer/expiry checked against Google's live public keys, and `email_verified` must be an explicit True (see §7, the bug this project fixes).
- **Replay of a price-change request**: mitigated by mandatory Idempotency-Key + optimistic version concurrency (§8).
- **MITM on a hostile network** (mobile-specific): mitigated by certificate pinning in the release `network_security_config.xml`; see `docs/mobile-security-checklist.md` §4 for the demonstration.

Database (`db.py`)

- **Tampering with historical records**: the audit table has BEFORE UPDATE/ BEFORE DELETE triggers that abort, AND every row is hash-chained to the previous row's hash, so even a change made by bypassing the triggers (e.g. a raw file-level edit) is detectable via `verify_log_integrity()` - this project's tests deliberately do that bypass and confirm detection (`tests/test_csrf_and_audit.py::test_audit_log_tamper_is_detected`).
- **Race conditions on price/stock**: see §8.

6. Key abuse cases and mitigations

Abuse case	Mitigation	Evidence
Customer crafts a request for /api/v1/products hoping to see cost/stock	products.visible_fields() never includes those keys in a customer's serialized response - not a UI hide	tests/test_rbac.py::test_customer_search_hides_cost_and_inactive
Vendor A edits vendor B's product by guessing its id	_owns_or_admin() check re-derives ownership server-side; returns 404 (not 403) to avoid confirming the id exists	tests/test_rbac.py::test_vendor_cannot_edit_another_vendors_product
Attacker double-clicks / retries a price-increase request	Idempotency-Key + version concurrency	docs/test-evidence/concurrency_price_race.py
Two admins change the same product's price at the same instant	Optimistic concurrency (version column), loser gets 409	same script, Test 1
Attacker registers with admin in the request body	register_user/API register hardcode role='customer'; role can only be changed via users_admin.update_user, admin-only	tests/test_rbac.py::test_admin_only_user_admin_endpoints
Attacker brute-forces a password	Lockout after 5 failures (auth_service.authenticate_password) + rate limiting	docs/test-evidence/rate_limiting_output.txt
Attacker who stole a password still can't log in	MFA is mandatory before any full session/token is issued	tests/test_mfa_and_google.py::test_password_only_login_cannot_reach_products
Attacker registers a Google account impersonating a real email that Google hasn't verified	validate_google_userinfo rejects unless email_verified is True (fail-closed)	tests/test_mfa_and_google.py::test_google_email_verified_missing_claim_is_rejected_fail_closed
Third-party site submits a hidden form to /reset-db or /admin/users/... against a logged-in admin	CSRF token required on every POST	tests/test_csrf_and_audit.py::test_web_post_without_csrf_token_is_rejected
Attacker on the same Wi-Fi as a mobile user runs a MITM proxy	Certificate pinning in the release network security config	docs/mobile-security-checklist.md \$4
Rooted device / stolen phone, attacker reads app-private storage	Tokens encrypted at rest (EncryptedSharedPreferences), allowBackup=false	docs/mobile-security-checklist.md \$3

7. Design decision: fixing the Google email_verified bug

The original lab's google_callback() did userinfo.get("email_verified", True) - if Google ever omitted the claim, the account was trusted anyway. This project treats "missing claim" and "email_verified: false" identically: both are rejected (security.validate_google_userinfo, fail-closed). The same function is now the *only* place either the Web callback or the Mobile endpoint decides this, so the policy can't drift between interfaces (see the OWASP table, A07 row).

8. Design decision: price protection under concurrency

Two independent, composable controls, both enforced inside one SQLite BEGIN IMMEDIATE transaction (products.update_price):

5. **Optimistic concurrency** (version column) - a writer must present the version it last read; if it's stale, the UPDATE matches zero rows and the writer gets a clean 409, never a silently lost update.

6. **Idempotency-Key** - the first request to acquire the write lock for a given key wins and its response is cached; every later request (retry, double-tap, or a genuinely concurrent duplicate) with the same key receives the identical cached response instead of re-applying the price change.

Real, reproducible concurrent-HTTP-request evidence for both is in docs/test-evidence/concurrency_price_race.py and its captured output.

9. Residual risk / accepted limitations

- SQLite is single-file/single-writer; the design accepts serialized writes (readers still proceed under WAL) as a deliberate tradeoff for a classroom-scale lab rather than introducing a separate DB service.
- The password-reset "email" is written to a local file (backend/mail_outbox/) instead of sent via real SMTP, per the assignment's "synthetic test data only" rule - this is not a production pattern and is documented as such in mail.py.
- LAB_ENC_KEY (MFA-secret-at-rest encryption) falls back to a key derived from LAB_SECRET_KEY if unset, purely so the lab still boots with zero setup; any real deployment must set both independently.
- A10 (SSRF) has no dedicated control because nothing in this app makes server-side requests to a caller-supplied URL; noted as not-applicable in the OWASP table rather than silently omitted.